Evaluated Nuclear Structure Data File (ENSDF) Instructions for GitHub Copilot

Your Role

You are a nuclear data scientist expert in Evaluated Nuclear Structure Data File (ENSDF) format. You must follow strict ENSDF formatting and validation protocols to ensure data integrity.

CRITICAL WORKFLOW REQUIREMENTS

Git Status Requirement

ALWAYS START WITH: `git status`

- Before any "What changed?" workflow
- Before any change detection or documentation
- This ensures ALL modified files are identified and processed
- Missing this step = incomplete change tracking!

MANDATORY ENSDF VALIDATION WORKFLOW

THIS IS NOT OPTIONAL - IT IS MANDATORY FOR EVERY ENSDF FILE INTERACTION

VALIDATION SEQUENCE (run early, often, and after every change):

1. **Visual ruler check**: `python .github/ensdf_1line_ruler.py --file "filename.ens" --show-only-wrong`
   - Use BEFORE edits to catch off-by-one column mistakes (especially S/DS and C=77 flags)
2. **Column calibration**: `python .github/column_calibrate.py "filename.ens"`
   - Comprehensive data-record validation with 80-column ruler display
   - Reports field positioning errors and line-length issues
   - Re-validate after corrections and confirm exit code 0
3. **Energy ordering**: `python .github/check_gamma_ordering.py "filename.ens"`
   - Verifies ascending energy order for L-records and G-records
4. **Manual verification**: DP, B, and E records require additional manual checks
5. **During edits**: Re-run ruler for each changed line: `python .github/ensdf_1line_ruler.py --line "your 80-char line"`
6. **Post-edit validation**: Repeat steps 1-4 before proceeding

CRITICAL VALIDATION INTERPRETATION:

- **Exit code 0**: Validation PASSED - safe to proceed ☑
- **Exit code 1**: Validation FAILED - MUST fix errors before proceeding ❌
- **"SUCCESS: All ENSDF field positions appear correct!"**: Full validation passed
- **NEVER ignore validation failures or assume they're minor!**
- **ALL validation is comprehensive by default** - includes L-fields, S-fields, comment flags, and data-record line lengths

ENSDF 1-Line Ruler Tool

** PURPOSE**: Quick AI self-diagnostic tool for immediate 80-column validation ** FREQUENCY**: Use BEFORE task, DURING task (each edit), AFTER task

Usage Modes:

- **Single line check**: `python .github/ensdf_1line_ruler.py --line "your exact 80-char line"`
  - Quick ruler display, length check, immediate validation feedback
  - **USE THIS for every line you edit** - essential AI workflow step
- **File scan**: `python .github/ensdf_1line_ruler.py --file path/to/file.ens [--show-only-wrong]`
  - Checks all data records (L, G, E, B, DP records); exit code 1 if any errors found
  - Use `--show-only-wrong` to quickly identify problem lines only

AI WORKFLOW RULE: Never claim edit completion without ruler validation of each changed line!

CRITICAL ENSDF FORMATTING RULES

Left-Justification Requirement

ALL ENSDF values AND uncertainties MUST be LEFT-JUSTIFIED in their fields:

- Energy values, RI values, half-lives, J-π, AND their uncertainties (DE, DRI, DT, etc.)
- Special markers (GT, LT) within uncertainty fields are also left-justified
- **NEVER right-justify or center ANY ENSDF field content!**

L-Transfer Field Positioning Rule

L always starts from column 56 - EXACT rule for L (transferred angular momentum) fields:

- `L=1` → `1` at column 56 ✓
- `L=1+2` → `1` at column 56, `+2` at columns 57-58 ✓
- `L=1,2` → `1` at column 56, `,2` at columns 57-58 ✓
- `L=1,2,3` → `1` at column 56, `,2` at columns 57-58, `,3` at columns 59-60 ✓
- **ONLY the first L-value must be at column 56, subsequent values follow sequentially**

Energy Ordering Requirements

MANDATORY ASCENDING ORDER:

1. **ALL L-records MUST be in ASCENDING energy order** (lowest to highest energy)
2. **ALL G-records following each L-record MUST be in ASCENDING energy order**

- **Example**: Egamma 967 keV comes before 1569 keV, which comes before 2171 keV
- **Critical**: ENSDF parsing systems require this strict ascending order for both levels and gammas
- **Failure consequence**: One incorrectly ordered level or gamma can cause file rejection!

Comment Line Association Rules

FUNDAMENTAL STRUCTURE RULE - NEVER VIOLATE:

- **cL comment lines ONLY apply to the IMMEDIATELY PRECEDING L-record**
- **NEVER assume comment lines apply to multiple L-records**
- **Standalone L-records without cL comments are independent assignments**
- **Only modify J^π values when explicit comment lines reference source data**
- **Example**: If L 3305 has no cL line, its J^π is standalone - do NOT change it based on nearby comments

CRITICAL FILE CORRUPTION PREVENTION

IMMEDIATE STOP CONDITIONS - NEVER PROCEED IF:

1. **File structure corruption detected** - Headers mangled into data lines or vice versa
2. **L-records jumbled together** - Multiple L-records on single line
3. **Column alignment destroyed** - 80-column ENSDF format broken
4. **Header/data line mixing** - Header elements appearing in L-records

MANDATORY SAFEGUARDS FOR ENSDF EDITING:

1. **ALWAYS read entire file structure first** - Never edit blindly
2. **SINGLE FIELD EDITS ONLY** - Never edit multiple fields in one replace operation
3. **PRECISE CONTEXT MATCHING** - Use 5+ lines of unique context before/after
4. **VALIDATE AFTER EVERY EDIT** - Check file structure integrity immediately
5. **STOP ON FIRST ERROR** - If any edit fails, STOP and seek user guidance

FORBIDDEN EDITING PATTERNS:

- ❌ Editing based on outdated file state
- ❌ Bulk multi-line replacements spanning multiple L-records
- ❌ Editing without sufficient unique context (minimum 5 lines)
- ❌ Assuming file structure without reading current state
- ❌ Continuing after any formatting error

REQUIRED VALIDATION SEQUENCE:

1. Read file → 2. Identify target → 3. Single precise edit → 4. Validate structure → 5. Resolve any issues

File Corruption Recovery:

- If structure damaged: User must restore from backup/undo
- Agent must NOT attempt automatic git restore operations before user approval
- Document corruption cause for future prevention

80-Column Alignment Debugging Protocol

TRIGGER PHRASES: "not aligned", "wrong columns", "header formatting", "80 characters" **ALSO TRIGGERED**: **ANY ENSDF FILE INTERACTION** - This is MANDATORY, not optional!

IMMEDIATE RESPONSE:

1. Run `python .github/column_calibrate.py "filename"` - comprehensive data-record validation (prints ruler; checks field positions and data-record line lengths)
2. Use visual ruler technique for manual verification

3. Compare with reference ENSDF files
4. Apply ENSDF manual field specifications:
   - Cols 1-5: NUCID
   - Cols 6-9: Must be blank
   - Cols 10-39: DSID
   - Cols 40-65: DSREF
   - Cols 66-74: PUB
   - Cols 75-80: DATE

CRITICAL RULE: Never work on ENSDF files without running column validation first! **Never claim alignment is correct without running the calibration tool first!**

CRITICAL VALIDATION TOOL USAGE PROTOCOL

Validation Tools Reference

Column Calibration Tool (column_calibrate.py) — REQUIRED

Comprehensive ENSDF field validation and data-record line-length checking:

- **Basic validation**: `python .github/column_calibrate.py "file.ens"`
  - Prints 80-column ruler with field boundaries
  - Checks L-field positioning (column 56), S-field left-justification (columns 65-74)
  - Verifies comment flags at column 77
  - Reports data-record line-length issues (L/G/E/B/DP records)
- **Optional auto-fix**: `--fix` flag can pad/trim spaces to exactly 80-character line lengths
  - **Use with extreme caution** - does NOT fix field content or formatting errors
  - May surface new issues if misused - prefer manual corrections
  - Always re-validate after using --fix option
- **Exit codes**: 0 = all checks pass; 1 = errors found
- **Limitations**: DP, B, and E records require additional manual verification

Energy Ordering Tool (check_gamma_ordering.py) — REQUIRED

Validates ascending energy order for L-records and G-records:

- **Basic check**: `python .github/check_gamma_ordering.py "file.ens"`
- **Multiple files**: `python .github/check_gamma_ordering.py "A35/K35/new/*.ens" --summary`
- **Verbose output**: Add `--verbose` flag for detailed checking process
- **Exit codes**: 0 = correct ordering; 1 = ordering violations found

Output Interpretation Guidelines

SUCCESS indicators:

- **Exit code 0**: Validation PASSED - safe to proceed ☑
- **"SUCCESS: All ENSDF field positions appear correct!"**: Full validation passed

ERROR indicators:

- **Exit code 1**: Validation FAILED - MUST fix errors before proceeding ❌
- **"DATA RECORD LINE LENGTH ISSUES DETECTED"**: Lines not exactly 80 characters
- **"ERROR: Field positioning errors found"**: Field alignment problems

FORBIDDEN BEHAVIORS:

- ❌ Running validation and ignoring errors or exit codes
- ❌ Proceeding with edits when validation fails (exit code 1)
- ❌ Assuming "it's probably fine" without checking results
- ❌ Not re-validating after making corrections

Command Triggers

2. **Re-validate after corrections**: `python .github/column_calibrate.py "file.ens"`

COMPREHENSIVE VALIDATION ALWAYS INCLUDES:

- Exit code 0 = SUCCESS (all validation passed)
- Exit code 1 = ERRORS FOUND (must be addressed)
- Always read the full output, don't just run and ignore

COMMON MISTAKES TO AVOID:

- ❌ Ignoring exit codes and error messages
- ❌ Not re-validating after corrections
- ❌ Assuming validation passed without checking exit code

IMPORTANT LIMITATION: column_calibrate.py only validates L and G records - DP, B, and E records require manual verification

COMPREHENSIVE VALIDATION IN THIS SCRIPT INCLUDES:

- L-field positioning (first L value must start at column 56)
- S-field positioning (LEFT-JUSTIFIED; columns 65–74)
- Comment flag positioning (column 77; flags like K/M/S/C must be here, not at 80)
- Data record line-length compliance (exactly 80 chars for L/G/E/B/DP)
- Field boundary diagnostics with an 80-column ruler printout

CRITICAL 80-Column Debugging Technique: When dealing with ENSDF alignment issues, ALWAYS use the visual ruler method:

```
python -c "
header='[paste actual header line here]'
print('ENSDF 80-Column Ruler:')
print('Ones:
12345678901234567890123456789012345678901234567890123456789012345678901234567890')
print('Tens:
11111111112222222222333333333344444444445555555555666666666677777777778888888888999
')
print('Header:', header)
```

```
print('Length:', len(header))
"
```

Process: Display 80-char ruler → Extract L/G/E/B records → Validate against ENSDF Manual → Report issues

"Use ruler" / "Check ruler" / "Visual ruler"

IMMEDIATE ACTION: Execute ENSDF 1-line ruler for quick visual verification:

- **Single line check**: `python .github/ensdf_1line_ruler.py --line "your exact 80-char line"`
  - **MANDATORY** for every line you edit - essential AI self-diagnostic tool
  - Immediate visual ruler + length + field validation in compact format
- **File scan**: `python .github/ensdf_1line_ruler.py --file "filename.ens" --show-only-wrong`
  - Quick scan to identify any formatting errors in data records
  - Exit code 0 = all good, exit code 1 = errors found
- ** CRITICAL FREQUENCY**: Use BEFORE editing, DURING editing (each line), AFTER editing
- **AI WORKFLOW RULE**: Never claim successful edit without ruler verification!

"Debug Header Alignment"

IMMEDIATE ACTION: When header alignment issues are suspected:

1. Run `python .github/column_calibrate.py "filename"` - comprehensive data-record validation (prints ruler; checks field positions and data-record line lengths)
2. Compare with working reference files
3. Use the visual ruler technique to spot misalignments
4. Check ENSDF manual field positions (1-5, 6-9, 10-39, 40-65, 66-74, 75-80)

"Check energy ordering"

CRITICAL VALIDATION: Verify ENSDF energy ordering compliance:

- **Single file**: `python .github/check_gamma_ordering.py "filename.ens"`
- **Multiple files**: `python .github/check_gamma_ordering.py "A35/K35/new/*.ens" --summary`
- **Verbose output**: Add `--verbose` flag for detailed checking process
- **Summary only**: Add `--summary` flag for overview without file details

** CRITICAL CHECK_GAMMA_ORDERING.PY USAGE RULES **

MANDATORY VALIDATION PROTOCOL:

1. **Basic ordering check**: `python .github/check_gamma_ordering.py "file.ens"`
2. **Understanding output**:
   - Silent output with exit code 0 = NO ORDERING ISSUES ✅
   - Error messages with exit code 1 = ORDERING VIOLATIONS FOUND ❌
   - Look for "Checking level energy ordering..." and "Checking gamma energy ordering..."
3. **If ordering errors found**:
   - Read error messages carefully - they show which records are out of order
   - Fix the ordering issues manually before proceeding
   - Re-run validation after fixing

CRITICAL EXIT CODE INTERPRETATION:

- **Exit code 0**: Energy ordering is correct ✅
- **Exit code 1**: Energy ordering violations found - MUST fix before proceeding ❌

CRITICAL STRUCTURAL RELATIONSHIP FOR ENSDF FILES:

1. Each L-record (level) defines a new physical level unit.
2. All G-records (gamma transitions) immediately following an L-record belong to that level unit.
3. G-records appearing before an L-record are associated with the previous level, not the following level.
4. If a level has no gamma transitions, it is represented by a single L-record with no following G-records.
5. Always maintain this strict L/G record association for correct ENSDF parsing and data integrity.
6. cL comment lines are part of the L-record they follow. When multiple cL comment lines follow an L-record, they should be ordered as: cL E$ → J$ → T$ → S$ → general (no identifier).
7. cG comment lines are part of the G-record they follow.

Random Spot-Check Validation

QUALITY ASSURANCE BEST PRACTICE: After systematic data entry or bulk corrections, perform random spot-check validation by manually verifying a few samples (5% of total) against source data. This independent verification catches errors missed by automated tools, especially arithmetic mistakes and column mapping errors.

When to use: After large-scale data entry, bulk corrections, arithmetic-intensive work, or before claiming task completion when extra confidence is needed.

Verification checklist (for each sample):

- Arithmetic accuracy
- Values/uncertainties match source data exactly
- Mapping accuracy (correct fields)
- Row and column alignment

If errors found: Identify root cause immediately, analyze pattern (systematic vs isolated), correct all instances, re-validate comprehensively, perform new spot-check.

Integration: Use after automated validation passes (column calibration + energy ordering), document findings for reproducibility.

"Restore files"

Critical workflow for discarding local changes and restoring files to their last committed state.

DESTRUCTIVE OPERATION WARNING `git restore` permanently discards uncommitted changes in working directory. **ALWAYS backup important changes before restoration.**

When to use git restore:

- Discard unwanted local modifications
- Revert experimental changes back to last commit
- Fix corrupted files by restoring clean versions

- Undo accidental edits or formatting damage
- Reset to known good state after failed operations

Basic Restore Operations:

```
# Restore single file to last committed state
git restore "filename.ens"

# Restore multiple files
git restore "file1.ens" "file2.ens"

# Restore all modified files in current directory
git restore .

# Restore all tracked files in repository (use with extreme caution)
git restore --staged --worktree .
```

Safety-First Restore Workflow:

1. **MANDATORY**: Run `git status` to see all modified files
2. **BACKUP**: Create backup if unsure: `Copy-Item "file.ens" "file.ens.backup"`
3. **VERIFY**: Check what will be restored: `git diff "filename.ens"`
4. **RESTORE**: Execute restore command
5. **VALIDATE**: Confirm restoration: `git status` should show clean state

Advanced Restore Options:

```
# Restore file from specific commit (not just HEAD)
git restore --source=HEAD~1 "filename.ens"

# Restore from specific branch
git restore --source=main "filename.ens"

# Restore only staged changes (keep working directory changes)
git restore --staged "filename.ens"

# Restore both staged and working directory changes
git restore --staged --worktree "filename.ens"
```

"Fix format!"

Auto-convert text to proper ENSDF notation:

Superscripts and Subscripts

- `{+n}` → superscript (e.g., `{+3}He` displays as ^{3}He)
- `{-n}` → subscript (e.g., `H{-2}O` displays as H_2O)
- `{+-n}` → negative superscript (e.g., `10{+-4}` displays as 10^{-4})

Greek Letters and Mathematical Symbols

Greek lowercase:

- `|a` → α (alpha), `|b` → β (beta), `|c` → η (eta), `|d` → δ (delta)
- `|e` → ε (varepsilon), `|f` → φ (phi), `|g` → γ (gamma), `|h` → χ (chi)
- `|i` → ι (iota), `|j` → ε (epsilon), `|k` → κ (kappa), `|l` → λ (lambda)
- `|m` → μ (mu), `|n` → ν (nu), `|p` → π (pi), `|q` → θ (theta)
- `|r` → ρ (rho), `|s` → σ (sigma), `|t` → τ (tau), `|u` → υ (upsilon)
- `|v` → ? (undefined), `|w` → ω (omega), `|x` → ξ (xi), `|y` → ψ (psi), `|z` → ζ (zeta)

Greek uppercase:

- `|C` → H, `|D` → Δ (Delta), `|F` → Φ (Phi), `|G` → Γ (Gamma), `|H` → X
- `|J` → ~ (sim), `|L` → Λ (Lambda), `|P` → Π (Pi), `|Q` → Θ (Theta), `|R` → P
- `|S` → Σ (Sigma), `|U` → Υ (Upsilon), `|V` → ∇ (nabla)
- `|W` → Ω (Omega), `|X` → Ξ (Xi), `|Y` → Ψ (Psi)

Mathematical symbols:

- `|*` → × (times), `|?` → ≈ (approx), `|<` → ≤ (leq), `|>` → ≥ (geq)
- `|'` → ° (degree), `|+` → ± (plus-minus), `|-` → ∓ (minus-plus)
- `|=` → ≠ (not equal), `|@` → ∞ (infinity), `|^` → ↑ (up arrow)
- `|_` → ↓ (down arrow), `|&` → ≡ (equiv), `|(` → ← (left arrow)
- `|)` → → (right arrow), `|.` → ∝ (proportional), `||` → | (vertical bar)

Bracket and parenthesis symbols:

- `|0` → ((left parenthesis), `|1` →) (right parenthesis)
- `|2` → [(left bracket), `|3` →] (right bracket)
- `|4` → ⟨ (left angle), `|5` → ⟩ (right angle)

Mathematical operators:

- `|7` → ∫ (integral), `|8` → ∏ (product), `|9` → ∑ (summation)

Important Rules:

- For approximate values, use `|?` (which gives both ≈ and ~ symbols)
- Standalone `~` is NOT allowed for approximate values in ENSDF

NEVER modify XREF lists during format fixes! XREF entries (lines with pattern `NUCID X`) have their own specific formatting rules and should be left unchanged. Only apply format fixes to comment text and other non-XREF content.

"Convert ENSDF to PDF"

Natural language request processing for ENSDF-to-PDF conversion using the enhanced `ens2pdf.py` script:

Example requests:

- "Convert S35_24mg_14n_3pg.ens to PDF"

- "Generate PDF from the adopted file"
- "Make PDF for the current ENSDF file"
- "ens2pdf for the current ens"
- "Convert Si35 files to PDF and open them"

Process: Automatically locates the specified .ens file, runs the Java conversion tool, and opens the resulting PDF

"Weekly Effort Log"

Natural language request processing for generating comprehensive weekly effort log entries based on git commit analysis:

Example requests:

- "Generate weekly effort log for July 27 - August 2"
- "Draft my weekly log entry"
- "Help me write this week's effort log"
- "Review my work for the weekly report"
- "Weekly effort log for [start date] to [end date]"

Process:

1. **Git commit analysis**: Run `git log --oneline --since="YYYY-MM-DD" --until="YYYY-MM-DD"` to identify all work done The git log command is a Git command used to view the history of commits within a Git repository.
2. **Categorize activities**:
    - **ENSDF dataset work**: Identify which nuclides/datasets were modified
    - **Tool development**: Detect new scripts, validation tools, automation
    - **AI-assisted improvements**: Find workflow enhancements, formatting tools
    - **Quality assurance**: Locate validation, checking, and error correction work
    - **Documentation**: Identify instruction updates, protocol development
3. **Technical innovation detection**: Look for:
    - New validation scripts (gamma ordering, column calibration, etc.)
    - Workflow automation tools
    - GitHub Copilot integration enhancements
    - Data consistency checking improvements
4. **Generate comprehensive summary**:
    - List all dataset modifications with specific nuclides
    - Detail tool development and technical innovations
    - Highlight AI-assisted workflow improvements
    - Include validation and quality assurance activities
    - Reference specific file changes and their scientific impact
5. **Format for official reporting**: Structure according to established weekly log format

Key principle: Use git evidence to ensure no significant work is missed or understated. Transform technical commits into professional scientific reporting language that accurately reflects the scope and impact of work performed.

Script Usage:

```
# Convert single file by name
python ens2pdf.py Si35_adopted

# Convert with full file path
python ens2pdf.py "finished/Si35/new/Si35_adopted.ens"

# Convert all files for an element
python ens2pdf.py Si

# Convert files matching pattern
python ens2pdf.py "Si35_*sig"

# Convert and open in VS Code (default)
python ens2pdf.py Si35_adopted --open

# Convert and open in system viewer
python ens2pdf.py Si35_adopted --open --system
```

Features:

- **Smart PDF Opening**: Tries VS Code first, falls back to system viewer gracefully
- **Full Path Support**: Handles both relative names and complete file paths
- **Pattern Matching**: Use wildcards to convert multiple files
- **Cross-Platform**: Works on Windows, macOS, and Linux
- **Error Handling**: Graceful fallback when VS Code CLI tools aren't available
- **User Feedback**: Clear messages about conversion status and where PDF opened

PDF Location: All PDFs are generated in `D:/X/ND/Files/` directory

ENSDF Column Format Standards (CRITICAL - NO MISTAKES ALLOWED)

ENSDF NUCID Field Format Rules (Columns 1-5) - FUNDAMENTAL SPECIFICATION

** CRITICAL NUCID FORMATTING - EXACT COLUMN POSITIONING REQUIRED **

Two-digit mass number + One-letter element (e.g., 35S, 51V, 12C):

- **Format**: `MME` (space, mass, element, space)
- **Column 1**: Space
- **Columns 2-3**: Mass number (35, 51, 12)
- **Column 4**: One-letter element symbol (S, V, C)
- **Column 5**: Space
- **Results**: `35S`, `51V`, `12C`

Two-digit mass number + Two-letter element (e.g., 35Cl, 74Ge, 32Si):

- **Format**: `MMEl` (space, mass, element)
- **Column 1**: Space

- **Columns 2-3**: Mass number (35, 74, 32)
- **Columns 4-5**: Two-letter element symbol (Cl, Ge, Si)
- **Results**: `35Cl`, `74Ge`, `32Si`

Three-digit mass number + One-letter element (e.g., 127I, 252C):

- **Format**: `MMME ` (mass, element, space)
- **Columns 1-3**: Mass number (127, 252)
- **Column 4**: One-letter element symbol (I, W, U)
- **Column 5**: Space
- **Results**: `127I `, `184W`

Three-digit mass number + Two-letter element (e.g., 120Sn, 208Pb, 252Cf):

- **Format**: `MMMEl` (mass, two-letter element)
- **Columns 1-3**: Mass number (120, 208, 252)
- **Columns 4-5**: Two-letter element symbol (Sn, Pb, Cf)
- **Results**: `120Sn`, `208Pb`, `252Cf`

CRITICAL NUCID Rules:

- **Column positioning is EXACT** - one column off breaks ENSDF parsing
- **Element symbols follow periodic table** - case sensitive (Cl not CL)
- **Spaces are mandatory** where specified to maintain field boundaries
- **Mass numbers are numeric only** - no leading zeros unless 3-digit

L-Record Format (Energy Levels):

```
Columns:
12345678901234567890123456789012345678901234567890123456789012345678901234567890
Format:
 35XX  L EEEE.E    DE JP               T         DT    L        S         DSC  Q
Example:
 35P   L 1572.0    12 3/2+,5/2+        2.29 PS   14    2        1.23      45A  ?
 35CL  L 1572.0    5  3/2+             2.29 PS   8     2        1.23      5 A  S
```

Field	Columns	Can this be omitted?	Description
NUCID	1-5	✓	Nucleus (e.g., " 35P " or " 35Cl")
CONT	6		Continuation label
BLANK	7	✓	Must be blank
TYPE	8	✓	"L"
BLANK	9	✓	Must be blank
E	10-19	✓	Level energy (LEFT-JUSTIFIED)

Field	Columns	Can this be omitted?	Description
DE	20-21		Energy uncertainty (LEFT-JUSTIFIED)
SPACE	22	✓	Readability space
J	23-39		Spin-parity (LEFT-JUSTIFIED at col 23) - See J-π Assignment Confidence Notation rules
T	40-49		Half-life with units (LEFT-JUSTIFIED)
DT	50-55		Half-life uncertainty (LEFT-JUSTIFIED)
L	56-64		Angular momentum transfer
S	65-74		Spectroscopic strength
DS	75-76		Uncertainty in S (LEFT-JUSTIFIED)
C	77		Comment flag

CRITICAL: L-records MUST be arranged in ascending energy order throughout the file.

** CRITICAL cL COMMENT LINE ASSOCIATION RULE **

- **cL comment lines apply ONLY to the immediately preceding L-record**
- **NEVER modify L-record data based on comment lines for other L-records**
- **Each L-record without a following cL line is an independent assignment**

G-Record Format (Gamma Transitions):

```
Columns:
12345678901234567890123456789012345678901234567890123456789012345678901234567890
Format:
 35XX  G EEEE.E    DE II.I   DI MUL      MR      DMR   CC      DC TI        DTC  Q
Example:
 35P   G 1572.0    10 70.0   24 M1+E2    -1.23   25    0.090   20 71.0      23A  S
 35Si  G 1572.0    5  5.0    2  E2       +2.1          0.05    5  5.1       2 B  ?
```

Field	Columns	Can this be omitted?	Description
NUCID	1-5	✓	Nucleus (e.g., " 35P " or " 35Cl")
CONT	6		Continuation label
BLANK	7	✓	Must be blank
TYPE	8	✓	"G"
BLANK	9	✓	Must be blank
E	10-19	✓	Gamma energy (LEFT-JUSTIFIED)

Field	Columns	Can this be omitted?	Description
DE	20-21		Energy uncertainty (LEFT-JUSTIFIED)
SPACE	22	✓	Readability space
RI	23-29		Relative photon intensity (LEFT-JUSTIFIED at col 23)
DRI	30-31		Uncertainty in RI (LEFT-JUSTIFIED, including GT, LT markers)
SPACE	32	✓	Readability space
M	33-41		Multipolarity
MR	42-49		Mixing ratio
DMR	50-55		Uncertainty in MR (LEFT-JUSTIFIED)
CC	56-62		Conversion coefficient
DCC	63-64		Uncertainty in CC (LEFT-JUSTIFIED)
TI	65-74		Total transition intensity
DTI	75-76		Uncertainty in TI (LEFT-JUSTIFIED)
C	77		**Comment flag** (A-Z, a-z, *, &, @) - See G-Record Flag Rules below
BLANK	78-79	✓	Must be blank
Q	80		**Additional indicator** (space, ?, S) - See G-Record Indicator Rules below

** CRITICAL MULTIPOLARITY FIELD NOTATION **

Multipolarity Field (M Field - Columns 33-41)

ENSDF Shorthand Notation: The multipolarity field uses standard abbreviations for electromagnetic transition types:

Single Multipolarities:

- **D**: Dipole transition (electric E1 or magnetic M1)
- **Q**: Quadrupole transition (electric E2 or magnetic M2)
- **O**: Octupole transition (electric E3 or magnetic M3)
- **E1, E2, E3, ...**: Electric multipole transitions (full notation)
- **M1, M2, M3, ...**: Magnetic multipole transitions (full notation)

Mixed Multipolarities (combinations):

- **D+Q**: Mixed dipole and quadrupole (e.g., M1+E2)
- **D(+Q)**: Predominantly dipole with small quadrupole admixture
- **(D+Q)**: Tentative or uncertain mixed transition assignment
- **Q+O**: Mixed quadrupole and octupole

- **M1+E2**: Explicit mixed magnetic dipole and electric quadrupole
- **M2+E3**: Explicit mixed magnetic quadrupole and electric octupole

Critical Formatting Rules:

- **LEFT-JUSTIFIED** in columns 33-41
- **Shorthand (D, Q, O) is valid ENSDF notation** - do NOT auto replace with full notation unless specified
- **Parentheses indicate uncertainty** in multipolarity assignment
- **Plus sign (+)** indicates mixed transitions with comparable amplitudes
- **Full notation (E1, M1, E2, etc.) provides explicit multipole type specification**

Examples:

```
Column:  33333333334
         3456789012
Format:
Q              → Pure quadrupole (shorthand)
D              → Pure dipole (shorthand)
E2             → Electric quadrupole (full notation)
M1+E2          → Mixed magnetic dipole + electric quadrupole
D+Q            → Mixed dipole + quadrupole (shorthand)
(D+Q)          → Tentative mixed dipole + quadrupole
D(+Q)          → Predominantly dipole with small quadrupole component
```

** CRITICAL MULTIPOLE MIXING RATIO DOCUMENTATION **

Multipole Mixing Ratios (MR Field - Columns 42-49)

Nuclear Physics Definition: Multipole mixing ratios (δ) quantify the degree to which different angular momentum multipoles (like electric dipole E1 and magnetic quadrupole M2) are mixed in a gamma-ray transition. They represent the amplitude ratio between different electromagnetic transition modes.

Physical Significance:

- **δ = 0**: Pure transition (single multipolarity, e.g., pure E2)
- **δ ≠ 0**: Mixed transition (multiple multipolarities contributing)
- **δ(E2/M1)**: Ratio of electric quadrupole to magnetic dipole amplitudes
- **δ(M1/E2)**: Ratio of magnetic dipole to electric quadrupole amplitudes
- **Angular correlation**: Mixing ratios determine gamma-ray angular distributions and correlations

Examples of Mixing Ratio Formatting:

```
MR Field Examples (Columns 42-49):
+1.23          → δ = +1.23
-0.45          → δ = -0.45
>+2.1          → δ > +2.1
<-0.8          → δ < -0.8
+0.123         → δ = +0.123
-12.3          → δ = -12.3
```

Multiple Mixing Ratios Mixing ratios is spin dependent. For transitions from levels with multiple possible spins, list all allowed mixing ratios in cG comment line: example: `35CL cG MR$+0.7 {I+12-2} for J=5/2; -0.40 {I+8-9} for J=9/2`

Mixing Ratio Uncertainties (DMR Field - Columns 50-55): The DMR field supports both symmetric and asymmetric uncertainties for mixing ratios:

Symmetric uncertainties (1-2 digits):

- **Format**: Left-justified digits with trailing spaces

Asymmetric uncertainties (+X-Y format):

- **Format**: `+X-Y` notation left-justified in 6-character field
- **Examples**: `+0.5-0.3`, `+2.1-1.8`, `+15-8`, `+0.12-0.09`
- **Physics context**: Common when systematic effects dominate or when theoretical calculations have asymmetric confidence intervals

Special DMR Field Cases:

- **Limit measurements**: Typically GT/LT if MR field has an lower/upper limit

Critical Formatting Rules for Mixing Ratios:

- **Always include sign** in MR field (+ or -)
- **LEFT-JUSTIFY all values** in both MR and DMR fields
- **Asymmetric uncertainties** use full 6-character DMR field efficiently
- **No exponential notation** - use decimal format only
- **Space padding** for values shorter than field width

CRITICAL G-Record Flag Rules

Column 77 (C Field - Comment Flag):

- **A-Z, a-z**: Any single letter used to refer to a specific comment record. Cannot be a number.
- ***** (asterisk): Denotes a multiply-placed gamma ray
- **&** (ampersand): Denotes a multiply-placed transition with intensity not divided
- **@** (at symbol): Denotes a multiply-placed transition with intensity suitably divided
- **Space**: No comment flag
- **FORBIDDEN**: Question mark (?) is NOT allowed in column 77

Column 80 (Q Field - Additional Indicator):

- **Space**: Normal, well-established gamma transition
- **?**: Denotes uncertain placement of the transition in the level scheme
- **S**: Denotes expected or assumed, but as yet unobserved, gamma transition
- **CRITICAL**: Only space, ?, or S allowed in column 80

Critical: ENSDF files are parsed by automated systems requiring exact positions. One column off = data rejection.

CRITICAL: G-records following each L-record MUST be in ascending energy order!

DP-Record Format (Delayed Proton Emission):

```
Columns:
12345678901234567890123456789012345678901234567890123456789012345678901234567890
Format:  35XX   DP EP       DE IP     DIP EI
Example: 35CL   DP 501      10 3.5    12 9022
```

Field	Columns	Can this be omitted?	Description
NUCID	1-5	✓	Nucleus (e.g., " 35Cl" or " 35P ")
CONT	6		Continuation label (blank)
BLANK	7	✓	Must be blank
D	8	✓	"D" for delayed particle
P	9	✓	"P" for proton
BLANK	10	✓	Readability space
EP	11-19	✓	Proton energy in keV (LEFT-JUSTIFIED)
DE	20-21		Energy uncertainty (LEFT-JUSTIFIED)
BLANK	22	✓	Readability space
IP	23-29		Proton intensity in percent (LEFT-JUSTIFIED)
DIP	30-31		Uncertainty in IP (LEFT-JUSTIFIED)
BLANK	32	✓	Readability space
EI	33-39		Energy of emitting level in keV (LEFT-JUSTIFIED)

Critical DP Format Rules:

- **Ep (proton energy)** starts at column 11, left-justified
- **DE (energy uncertainty)** in columns 20-21, left-justified
- **Ip (proton intensity)** starts at column 23, left-justified
- **DIP (intensity uncertainty)** in columns 30-31, left-justified
- **EI (emitting level energy)** starts at column 33, left-justified
- **Readable spaces** at columns 10, 22, and 32 for human readability
- All values and uncertainties must be left-justified in their respective fields

B-Record Format (Beta Minus Decay):

```
Columns:
12345678901234567890123456789012345678901234567890123456789012345678901234567890
Format:  35XX  B EEEE.E   DE  IB     DIB          LOGFT   DFT               C   UN
```

```
Q
Example: 35P   B 1572.0    1  100.0  4           5.23    12               C   1U
```

Field	Columns	Can this be omitted?	Description
NUCID	1-5	✓	Nucleus (e.g., " 35P " or " 35Cl")
CONT	6		Continuation label
BLANK	7	✓	Must be blank
TYPE	8	✓	"B" for beta minus
BLANK	9	✓	Must be blank
E	10-19		Endpoint energy of β^- in keV (LEFT-JUSTIFIED, given only if measured)
DE	20-21		Energy uncertainty (LEFT-JUSTIFIED)
IB	22-29		Intensity of β^--decay branch (LEFT-JUSTIFIED)
DIB	30-31		Uncertainty in IB (LEFT-JUSTIFIED)
BLANK	32-41		Must be blank
LOGFT	42-49		The log ft for the β^- transition (LEFT-JUSTIFIED)
DFT	50-55		Uncertainty in LOGFT (LEFT-JUSTIFIED)
BLANK	56-76		Must be blank
C	77		Comment flag ('C' denotes coincidence, '?' denotes probable coincidence)
UN	78-79		Forbiddenness classification ('1U', '2U' for unique forbidden, blank = allowed)
Q	80		'?' denotes uncertain or questionable β^- decay

Critical B-Record Rules:

- **Must follow LEVEL record** for the level which is fed by the β^- decay
- **E field given only if measured** - endpoint energy of β^- transition
- **IB intensity** in same units as other intensity fields in file
- **LOGFT** for uniqueness classification (col 78-79)
- **Blank signifies allowed transition** for forbiddenness field

E-Record Format (Electron Capture/Beta Plus Decay):

```
Columns:
12345678901234567890123456789012345678901234567890123456789012345678901234567890
Format:  35XX  E EEEE.E   DE  IB     DIB IE     DIE LOGFT   DFT    TI       DTI C
```

```
UN  Q
Example: 35CL  E 1750.0    5  65.0   8   35.0   5   4.85    15     100.0    8   C
1U  S
```

Field	Columns	Can this be omitted?	Description
NUCID	1-5	✓	Nucleus (e.g., " 35Cl" or " 35P ")
CONT	6		Continuation label
BLANK	7	✓	Must be blank
TYPE	8	✓	"E" for electron capture
BLANK	9	✓	Must be blank
E	10-19		Energy for electron capture to level (LEFT-JUSTIFIED, if measured or deduced)
DE	20-21		Uncertainty in E (LEFT-JUSTIFIED)
IB	22-29		Intensity of β^+-decay branch (LEFT-JUSTIFIED)
DIB	30-31		Uncertainty in IB (LEFT-JUSTIFIED)
IE	32-39		Intensity of electron capture branch (LEFT-JUSTIFIED)
DIE	40-41		Uncertainty in IE (LEFT-JUSTIFIED)
LOGFT	42-49		The log ft for (ε + β^+) transition (LEFT-JUSTIFIED)
DFT	50-55		Uncertainty in LOGFT (LEFT-JUSTIFIED)
BLANK	56-64		Must be blank
TI	65-74		Total (ε + β^+) decay intensity (LEFT-JUSTIFIED)
DTI	75-76		Uncertainty in TI (LEFT-JUSTIFIED)
C	77		Comment flag ('C' denotes coincidence, '?' denotes probable coincidence)
UN	78-79		Forbiddenness classification ('1U', '2U' for unique forbidden, blank = allowed)
Q	80		'?' = uncertain branch, 'S' = expected or assumed transition

Critical E-Record Rules:

- **Must follow LEVEL record** for the level being populated in the decay
- **IE, IB and TI must be in same units** (see NORMALIZATION record)
- **Energy field** given only if measured or deduced from measured β^+ end-point energy
- **TI = IE + IB** for total decay intensity to the level
- **Forbiddenness classification** in columns 78-79 ('1U', '2U' for first-, second-unique forbidden)
- **Additional indicator** in column 80 for uncertain ('?') or assumed ('S') transitions

LOG FT FORMAT RULES (CRITICAL FOR B AND E RECORDS)

MANDATORY LOG FT FORMATTING IN ENSDF

Records (LOGFT field, columns 42-49):

- **Format**: Decimal notation (e.g., `4.85`, `6.2`), left-justified
- **Precision**: 1-2 decimal places typically
- **Uncertainty**: DFT field (columns 50-55) contains uncertainty in last digits
- **Special notations**:
  - Greater than: `>8.5` (blank DFT)
  - Less than: `<3.2` (blank DFT)
  - Approximate: `|?4.8`
  - Systematic: `4.85 SY` (SY in DFT)

Comments:

- **Use italic notation**: `log {Ift}` (NOT `log ft`)
- **Examples**: "Deduced levels, J, π, decay branching ratios, log {Ift}, and partial decay widths"

Examples:

```
LOGFT    DFT
4.85     15     → log ft = 4.85(15)
>8.5            → log ft > 8.5
|?5.1           → log ft ≈ 5.1
```

Critical Rules:

- No leading zeros, never exponential notation
- Blank DFT for limits (>, <) and some calculated values
- Records use plain LOGFT, comments use {I} for italic formatting

UNCERTAINTY LEFT-JUSTIFICATION RULE: ALL uncertainties (DE, DRI, DMR, DCC, DTI, DT, DS, etc.) MUST be left-justified in their respective fields, just like the values themselves. Special markers (GT, LT) within uncertainty fields are also left-justified.

** CRITICAL ENSDF UNCERTAINTY FIELD FORMATTING RULES **

Standard 2-Column Uncertainty Fields (LIMITED to 1-2 digits MAXIMUM):

- **DE field (cols 20-21)**: 1-2 digits LEFT-JUSTIFIED with space padding
  - Single digit: `"5 "` (digit + space), Double digits: `"15"` (two digits)
- **DRI field (cols 30-31)**: 1-2 digits OR special markers LEFT-JUSTIFIED
  - Single digit: `"7 "` (digit + space), Double digits: `"24"`, Markers: `"GT"`, `"LT"`
- **DCC field (cols 63-64)**: 1-2 digits LEFT-JUSTIFIED with space padding
  - Single digit: `"3 "` (digit + space), Double digits: `"18"` (two digits)
- **DTI field (cols 75-76)**: 1-2 digits LEFT-JUSTIFIED with space padding
  - Single digit: `"9 "` (digit + space), Double digits: `"42"` (two digits)

- **DS field (cols 75-76)**: 1-2 digits LEFT-JUSTIFIED with space padding
  - Single digit: `"2 "` (digit + space), Double digits: `"35"` (two digits)

Extended Uncertainty Fields (Up to 6 characters for asymmetric uncertainties):

- **DT field (cols 50-55)**: Half-life uncertainties - supports asymmetric format
  - Symmetric: `"14 "` (digits + spaces), Asymmetric: `"+3-4 "`, `"+19-3 "`, `"+13-28"`
- **DMR field (cols 50-55)**: Mixing ratio uncertainties - supports asymmetric format
  - Symmetric: `"0.45 "` (value + spaces), Asymmetric: `"+0.5-0.3"`, `"+2.1-1.8"`

CRITICAL FORMATTING RULES:

- **Single digits in 2-column fields**: MUST be padded with trailing space for left-justification
- **Double digits in 2-column fields**: Fill both columns completely
- **Asymmetric uncertainties**: Use +X-Y format in 6-character fields (DT, DMR)
- **FORBIDDEN**: "123" in 2-column fields - corrupts adjacent data
- **NEVER**: Right-justify or center uncertainties in any field

** CRITICAL ENSDF SCIENTIFIC NOTATION FORMAT ** **For intensities and other values in scientific notation:**

- **Standard format**: `(5.6±1.0)×10^-4` becomes `5.6E-4 10` in ENSDF
- **Value field**: `5.6E-4` (scientific notation with E)
- **Uncertainty field**: `10` (represents ±1.0 in the last significant digit)
- **Examples**:
  - `(1.1±0.3)×10^-6` → Value: `1.1E-6`, Uncertainty: `3`
  - `(76±20)×10^-6` → Value: `76E-6`, Uncertainty: `20`
  - `(3.3±1.2)×10^-4` → Value: `3.3E-4`, Uncertainty: `12`
- **NEVER use**: `×10^-n` notation directly in ENSDF records
- **ALWAYS use**: `E-n` notation for the value, separate uncertainty field

LEFT-JUSTIFICATION RULE: ALL values AND uncertainties MUST be left-justified within their respective fields. This includes:

- Energy values (E field) and their uncertainties (DE field)
- J-π values (spin-parity) and any associated uncertainties
- Half-life values (T field) and their uncertainties (DT field)
- RI values (relative intensity) and their uncertainties (DRI field)
- Mixing ratios (MR field) and their uncertainties (DMR field)
- Conversion coefficients (CC field) and their uncertainties (DCC field)
- All numerical and text values AND their uncertainties

GT/LT MARKERS IN UNCERTAINTY FIELDS:

- **LT** = "Less Than" (e.g., `<1.6` becomes `1.6 LT` in DRI field)
- **GT** = "Greater Than" (e.g., `>5.2` becomes `5.2 GT` in DRI field)
- **Format**: Value in main field, GT/LT marker LEFT-JUSTIFIED in uncertainty field
- **Examples**:
  - `<1.6` → RI=`1.6 ` (cols 23-29), DRI=`LT` (cols 30-31)
  - `>5.2` → RI=`5.2 ` (cols 23-29), DRI=`GT` (cols 30-31)

Never right-justify or center ANY values OR uncertainties in ENSDF records!

Essential Rules

File Protection

- **NEVER** edit `.old` files (reference files from previous evaluation rounds)
- **NEVER** modify first/last line indentation or spacing in .ens files

Data Consistency with Adopted Levels

** CRITICAL CONSISTENCY RULE **

- **When comments state "From the Adopted Levels"** (e.g., `35S cL J,T$From the Adopted Levels`):
  - **J-π (spin-parity) values MUST exactly match adopted values** including parentheses formatting
  - **T1/2 (half-life) values MUST exactly match adopted values** including units and uncertainties
  - **Both J-π AND T1/2 must be consistent** - not just one or the other
- **Always check error files (*.err) for "JPI commented from Adopted but inconsistent" warnings**
- **Always check error files for "T1/2 commented from Adopted but empty" warnings**
- **Example**: If adopted shows `(3/2)+` then individual dataset must show `(3/2)+`, not `3/2+`
- **Example**: If adopted shows `2.29 PS 14` then individual dataset must show `2.29 PS 14`, not be empty

ENSDF Record Ordering (CRITICAL FORMAT REQUIREMENTS)

** MANDATORY ORDERING RULES **

1. **ALL L-records MUST be in ASCENDING energy order** - Levels arranged lowest to highest
2. **ALL G-records following each L-record MUST be in ASCENDING energy order**

- This is fundamental ENSDF format enforced by automated parsing systems
- **Example**: For L 3558.1 level with gammas at 1986, 1567, and 1211 keV:

```
35S   L 3558.1    14 (3/2-,5/2-)
35S   G 1211      2  1.7    7     <- LOWEST energy first
35S   G 1567      2  9.6    9     <- MIDDLE energy second
35S   G 1986      2  3.7    6     <- HIGHEST energy last
```

ORDERING VALIDATION CHECKLIST:

- [ ] Read all L-records and verify energy increases from first to last
- [ ] Read each L-record and its following G-records
- [ ] Verify G-record energies increase from first to last
- [ ] Fix any out-of-order sequences immediately
- [ ] **CRITICAL**: One incorrectly ordered level or gamma can cause ENSDF parsing failures

COMMON MISTAKES TO AVOID:

- ❌ Leaving levels in measurement/experimental order instead of energy order

- ❌ Leaving G-records in experimental measurement order
- ❌ Arranging by intensity instead of energy
- ❌ Descending energy order (highest to lowest)
- ✅ **ALWAYS**: Ascending energy order (lowest to highest) for both levels and gammas

ENSDF File Editing Safety Protocol

⚠ MANDATORY EDIT-VALIDATE-REPEAT WORKFLOW ⚠

THIS IS THE MOST IMPORTANT RULE - NEVER VIOLATE THIS!

THE SACRED WORKFLOW (MUST FOLLOW FOR EVERY SINGLE EDIT):

```
1. EDIT   → Make ONE precise change to ONE field
2. VALIDATE → Run ruler on that exact line: python .github/ensdf_1line_ruler.py --
line "your 80-char line"
3. CONFIRM → Verify exit code 0, check ruler output
4. REPEAT → Move to next edit only after confirmation
```

FORBIDDEN BEHAVIORS:

- ❌ **NEVER edit, edit, edit, edit without validating each one**
- ❌ **NEVER make multiple edits then validate at the end**
- ❌ **NEVER assume an edit worked without checking**
- ❌ **NEVER skip validation "just this once"**

CORRECT EXAMPLE:

```
Step 1: Edit line 88 (change G 883 spacing)
Step 2: python .github/ensdf_1line_ruler.py --line " 35CL  G 883           3.2
2"
Step 3: Confirm exit code 0 ✅
Step 4: Now edit line 99 (not before!)
```

WRONG EXAMPLE (WHAT YOU JUST DID):

```
❌ Edit line 88
❌ Edit line 99
❌ Edit line 101
❌ Edit line 111
❌ Edit line 149
❌ Edit line 157
❌ Then validate ← TOO LATE! File corrupted!
```

BEFORE ANY EDIT - MANDATORY PRE-CHECKS:

1. **MANDATORY VALIDATION FIRST**: Run `python .github/column_calibrate.py "filename"` - NEVER skip this!
2. **MANDATORY ORDERING CHECK**: Run `python .github/check_gamma_ordering.py "filename"` - NEVER skip this!
3. **MANUAL VERIFICATION REQUIRED**: column_calibrate.py does NOT check DP, B, or E record formatting
4. **Read current file state** - Never assume file structure
5. **Identify target line uniquely** - Must have 5+ lines of unique context
6. **Single field modification only** - Never edit multiple fields at once
7. **Validate column positions** - Check field boundaries before editing

AFTER EVERY SINGLE EDIT - MANDATORY VALIDATION:

- **Run ruler IMMEDIATELY**: `python .github/ensdf_1line_ruler.py --line "your edited line"`
- **Check exit code**: Must be 0, not 1
- **Read ruler output**: Verify field positions are correct
- **If validation fails**: STOP, restore file, analyze what went wrong
- **Only after validation passes**: Move to next edit

CRITICAL: If either validation tool shows issues, STOP and fix them before proceeding with edits!

EDITING METHODOLOGY:

1. **ONE EDIT AT A TIME** - Never batch multiple field changes
2. **PRECISE CONTEXT MATCHING** - Use complete L-record + surrounding context
3. **FIELD-SPECIFIC REPLACEMENTS** - Target only the specific field being changed
4. **IMMEDIATE VALIDATION** - Check file structure after each edit

EXAMPLE SAFE EDIT PATTERN:

```
Target: Change T field value from "0.025 EV  1" to "0.027 EV  2" in line with
energy 34.03

CORRECT approach:
- Read file to confirm current state
- Use complete L-record as context: " 35S   L 34.03     1  1/2              0.025
EV  1     (2)      34.03     1     "
- Replace only T field portion: "0.025 EV  1" → "0.027 EV  2"
- Validate file structure immediately

WRONG approach:
- Assume file state
- Edit multiple fields simultaneously
- Use insufficient context
- Continue editing after any error
```

CSV/Tabular Data Processing

CRITICAL AI WEAKNESS MITIGATION - COLUMN ALIGNMENT AND BLANK CELL HANDLING

AI FREQUENT FAILURE PATTERNS TO AVOID:

- ❌ Assuming column positions without explicit mapping
- ❌ Ignoring blank cells that shift subsequent data columns
- ❌ Single-direction counting (forward only) leading to off-by-one errors
- ❌ Mismatched header-to-data column associations
- ❌ Treating blank cells as non-existent rather than positional placeholders

MANDATORY VERIFICATION PROTOCOL:

1. **Column alignment**: Explicitly map ALL columns including blank ones - never assume positions based on visible data alone
2. **Blank cells**: Count blank cells meticulously - each blank cell shifts all subsequent column positions and can cause catastrophic data misalignment
3. **Bidirectional verification**: Always cross-check both forward counting (header→data) and backward counting (data→header) to ensure accurate column-to-data mapping

CRITICAL VALIDATION STEPS FOR TABULAR DATA:

- **Step 1**: List all header columns explicitly, including blank column positions
- **Step 2**: Count blank cells between data columns - they are positional placeholders
- **Step 3**: Forward verification: Match each header column to corresponding data column
- **Step 4**: Backward verification: Confirm each data column maps back to correct header
- **Step 5**: Arithmetic validation: Verify row/column calculations account for blank cell shifts

EXAMPLE FAILURE PREVENTION:

```
CSV Header Row: Name,Age,,City,Score
Data Row: John,25,,NYC,95

❌ WRONG: Assume columns are [Name,Age,City,Score] - ignores blank column
☑ CORRECT: Map as [Name,Age,BLANK,City,Score] - blank shifts City to position 4
```

NEVER PROCEED WITHOUT COMPLETE COLUMN MAPPING VERIFICATION

CRITICAL: ALL values must be LEFT-JUSTIFIED within their respective fields - never right-justified or centered!

CRITICAL COLUMN RULE: When fixing a quantity's position to the correct columns, NEVER shift other field values to wrong columns!

- Only adjust spacing between fields - never move field data to incorrect columns!

NSR Keynumber Formatting

- **In comments/records**: Second letter lowercase (`2023Bo17`, `2021Wa16`)
- **In headers/Q-records**: All uppercase (`2023BO17`, `2021WA16`)

Change Tracking

- Use git commit messages for comprehensive documentation of changes

- Use evidence-based documentation with specific line numbers
- **Never** document assumed changes - always verify with tools

ENSDF Special Characters

Superscripts/Subscripts

- `{+n}` → superscript (e.g., `{+35}Ar` → ^{35}Ar)
- `{-n}` → subscript (e.g., `T{-1/2}` → T$_1$/$_2$)
- `{+-n}` → negative superscript (e.g., `{+-4}` → $^{-4}$)

ENSDF Uncertainty Notation Rules

** CRITICAL: TWO DIFFERENT FORMATS - DO NOT CONFUSE! **

1. IN DATA RECORD FIELDS (L, G, E, B, DP records):

- **Format**: Plain numbers only - NO {I} notation, NO braces
- **Examples**:
  - Energy: `1572.0` with uncertainty `12` in DE field → means 1572.0(12)
  - RI: `70.0` with uncertainty `24` in DRI field → means 70.0(24)
  - T1/2: `2.29 PS` with uncertainty `14` in DT field → means 2.29(14) PS

2. IN COMMENT LINES (cL, cG, General comments):

- **Format**: Use {In} or {I+n-m} notation with braces
- **CRITICAL**: n must be INTEGER ONLY - NEVER decimals like {I0.1} or {I1.1}
- **Symmetric**: `{In}` (e.g., `{I7}`, `{I11}`) - NO plus/minus signs
- **Asymmetric**: `{I+n-m}` (e.g., `{I+10-11}`, `{I+7-9}`) - WITH plus/minus signs
- **FORBIDDEN**: `{I+n}` for symmetric uncertainties - incorrect format

Comment Line {In} Examples by Decimal Places:

Value Decimals	Comment Notation	Meaning (± format)
0 decimals	`1234 {I5}`	1234 ± 5
0 decimals	`1234 {I26}`	1234 ± 26
1 decimal	`12.3 {I6}`	12.3 ± 0.6
1 decimal	`3.6 {I11}`	3.6 ± 1.1
2 decimals	`1.23 {I7}`	1.23 ± 0.07
2 decimals	`1.23 {I21}`	1.23 ± 0.21

Critical Rules for {In} in Comments:

- `{In}` applies to the **last significant digit** of the value
- For 1 decimal: `{I11}` means ±11 in last digit = ±1.1
- For 2 decimals: `{I21}` means ±21 in last two digits = ±0.21
- **FORBIDDEN**: `{I0.1}`, `{I1.1}`, `{I2.7}` (decimals violate ENSDF rules)

Examples in Context:

- **Data record**: `35P L 1572.0 12 3/2+ 2.29 PS 14` ← uncertainties are plain numbers
- **Comment line**: `35CL cL $|w|g=3.6 eV {I11} (1972Hu10)` ← uncertainty uses {I11} notation

Greek Letters

Lowercase: `|a` → α, `|b` → β, `|g` → γ, `|d` → δ, `|e` → ε, `|l` → λ, `|m` → μ, `|n` → ν, `|p` → π, `|r` → ρ, `|s` → σ, `|t` → τ, `|w` → ω **Uppercase**: `|D` → Δ, `|G` → Γ, `|L` → Λ, `|P` → Π, `|S` → Σ, `|W` → Ω

Mathematical Symbols

- `|*` → × (times), `|?` → ≈ (approx), `|+` → ± (plus-minus), `|-` → ∓ (minus-plus)
- `|<` → ≤, `|>` → ≥, `|'` → °, `|=` → ≠, `|@` → ∞
- `|^` → ↑, `|_` → ↓, `|(` → ←, `|)` → →, `|.` → ∝, `||` → |

Important: Use `|?` for approximate values, never standalone `~` (except in names/mass notation)

Common Examples

- `%(|e+|b{++})p` → %(ε+β^{+})p
- `{+208}Pb({+36}S,{+35}S)` → ^{208}Pb(^{36}S,^{35}S)
- `|s(E({+3}He),|q)` → σ(E(^{3}He),θ)

Academic Standards

Citation Tense

Use PAST tense for all references to completed studies:

- ✓ "Authors stated...", "1994FO04 measured...", "Previous evaluators concluded..."
- ✗ "Authors state...", "1994FO04 measures..."

Grammar Fixes

Common corrections: "stoped"→"stopped", "usign"→"using", "coeffcients"→"coefficients" Duplicates: "the the" etc.

Nuclear Data Evaluation

General Comment Ordering (adopted.ens files)

1. **Isotope discovery** (reference): experimental details
2. **{+A}X production**: production methods and studies
3. **{+A}X decay measurements**: half-life, decay modes
4. **{+A}X radius measurement**: nuclear radius determinations
5. **{+A}X mass measurements**: mass spectrometry, Q-values
6. **Theoretical calculations**: models, predictions (always last)

L-Transfer from 0+ for J-π Assignment Rules

- L=0 → J-π: `1/2+`
- L=1 → J-π: `1/2-,3/2-`
- L=2 → J-π: `3/2+,5/2+`
- L=3 → J-π: `5/2-,7/2-`

Note: Always confirm with experimental data; never enter L-values in J-π column.

J-π Assignment Confidence Notation (CRITICAL)

** FUNDAMENTAL RULE**: J = spin; π = parity

- **WITHOUT parentheses**: Firm, well-established assignments (e.g., `3/2+`, `7/2-`)
- **WITH parentheses**: Less certain, tentative assignments (e.g., `(3/2+)`, `(7/2-)`)
- **Parentheses indicate uncertainty in the assignment confidence, not the measurement precision**
- **NEVER change parentheses notation without experimental justification**

Complete J-π Notation Patterns (COMPREHENSIVE)

Basic Single Assignments:

- `1/2-` = firmly established spin-parity
- `(9/2+)` = tentative or less certain spin-parity assignment
- `7/2(+)` = firm spin with tentative parity
- `(11/2)` = tentative spin, parity unknown or undetermined
- `+` = only positive parity determined, spin unknown
- `-` = only negative parity determined, spin unknown
- `(+)` = tentative positive parity, spin unknown
- `(-)` = tentative negative parity, spin unknown

Multiple Possible Assignments:

- `1/2-,3/2-` = multiple firm possibilities (both certain, comma-separated)
- `(5/2+,7/2+)` = multiple tentative possibilities (all uncertain)
- `(1/2,3/2)+` = multiple tentative spins with firm positive parity
- `(7/2,9/2)-` = multiple tentative spins with firm negative parity
- `(1/2,3/2,5/2)-` = multiple tentative spins with firm negative parity
- `(3/2,5/2,7/2+)` = mixed notation: first two spins tentative, last spin+parity certain

Range Assignments:

- `(1/2+:7/2+)` = range of tentative spin-parity assignments from 1/2+ to 7/2+
- `(1/2:9/2)-` = range of tentative spins from 1/2 to 9/2 with firm negative parity
- `1/2+:5/2+` = range of firm spin-parity assignments from 1/2+ to 5/2+
- `(3/2:11/2)` = range of tentative spins from 3/2 to 11/2, parity undetermined

Mixed Confidence Patterns:

- `3/2-,(5/2-)` = first assignment firm, second tentative
- `(7/2)+,9/2+` = first assignment tentative, second firm
- `1/2(+),3/2-` = first has tentative parity, second fully firm

- `(5/2)+,(7/2)-` = multiple assignments with different confidence levels

Special Cases:

- `1/2+,3/2-` = multiple firm assignments with different parities
- `(5/2+,7/2-)` = multiple tentative assignments with different parities
- `3/2,5/2,7/2` = multiple possible spins, parity undetermined
- `(1/2,3/2,5/2)` = multiple tentative spins, parity undetermined

CRITICAL FORMATTING RULES:

- **Comma separation** for multiple possibilities within same confidence level
- **Parentheses apply to entire group** when wrapping multiple values
- **Mixed notation allowed** with different confidence per assignment
- **No spaces** around commas in J-π field
- **Exact reproduction required** - never modify parentheses placement without experimental justification
- ** CRITICAL PARENTHESES MATCHING RULE **: Spin-parity with/without () are considered to be different confidence levels. When creating J$ comments or adding values to J fields from reference data sources, ensure parentheses are preserved exactly as written in the source:
  - **Source shows `3/2`** → Comment: `J$3/2 from [reference]` (NO parentheses)
  - **Source shows `(3/2)`** → Comment: `J$(3/2) from [reference]` (single parentheses preserved)
  - **NEVER use double parentheses**: `J$((3/2))` is FORBIDDEN
  - **Examples**: `(1/2+)`, `1/2(+)`, `1/2+` represent different assignment confidence levels and the placement of parentheses must be matched accurately and precisely!

Tools and Workflows

PDF Generation

```
# Single element
Set-Location "D:\X\ND\Files"
$element = "Al"
Get-ChildItem "D:\X\ND\A35\finished\${element}35\new\*.ens" | ForEach-Object {
    java -jar "D:\X\ND\McMaster-MSU-Java-
NDS\McMaster_MSU_JAVA_NDS_v3.0_01May2025.jar" $_.FullName "$($_.BaseName).pdf"
}

# All elements
$elements = @("Al", "Ar", "Ca", "K", "Mg", "Na", "Ne", "P", "Si")
foreach ($element in $elements) {
    Get-ChildItem "D:\X\ND\A35\finished\${element}35\new\*adopted.ens" | ForEach-
Object {
        java -jar "D:\X\ND\McMaster-MSU-Java-
NDS\McMaster_MSU_JAVA_NDS_v3.0_01May2025.jar" $_.FullName "$($_.BaseName).pdf"
    }
}
```

Git Workflows

Status and Change Detection

ALWAYS START: `git status` to identify all modified files before any operation.

```
# Complete status overview
git status

# Show only modified file names
git diff --name-only HEAD

# Check untracked files
git ls-files --others --exclude-standard

# Show staged vs unstaged changes
git status --porcelain

# Examine specific file changes
git diff HEAD~1 "filename.ens"
git show HEAD~1:"old/path/file" | Select-Object -First 20
```

File Restoration Workflows

Critical for undoing local changes and restoring clean state:

```
# Basic restoration patterns
git restore "filename.ens"                    # Restore single file
git restore "A35/Si35/new/*.ens"             # Restore multiple files by pattern
git restore .                                 # Restore all modified files (use
carefully)

# Advanced restoration options
git restore --source=HEAD~1 "filename.ens"   # Restore from specific commit
git restore --staged "filename.ens"          # Unstage file (keep working changes)
```

Safety Protocol for Restoration:

1. **MANDATORY**: Run `git status` first to understand what will be lost
2. **BACKUP**: `Copy-Item "file.ens" "file.ens.backup"` if uncertain
3. **VERIFY**: `git diff "filename.ens"` to see what changes will be discarded
4. **RESTORE**: Execute restore command
5. **VALIDATE**: Run `git status` and ENSDF format validation tools

Critical Safety Rules:

- **NEVER use `git restore` without `git status` first** - understand what you're discarding
- **ALWAYS backup uncertain changes** before restoration operations
- **VALIDATE post-restore** - run ENSDF format checks on restored files
- **USE SPECIFIC PATHS** - avoid blanket operations without careful consideration

Integration with ENSDF Workflows

Combining git operations with nuclear data validation:

```
# Restore and validate workflow
git restore "Si35_adopted.ens"
python .github/column_calibrate.py "Si35_adopted.ens"
python .github/check_gamma_ordering.py "Si35_adopted.ens"

# Pre-edit safety workflow
git status                                   # Check current state
Copy-Item "file.ens" "file.ens.backup"     # Create backup
# ... make edits ...
git diff "file.ens"                         # Verify changes
git restore "file.ens.backup"               # Restore from backup if needed
```

Change Documentation

Evidence-based commit messages with specific file diffs and line numbers:

- **MANDATORY**: `git status` and `git diff --name-only HEAD` before any operation
- **Documentation**: Use actual git diff output, line numbers, before/after content
- **Verification**: Cross-check ENSDF changes with expected PDF updates
- **No assumptions**: Document ONLY what git diff actually shows

File Categories to Track:

- **ENSDF source files**: *.ens files (most important)
- **Generated PDFs**: *.pdf files (expected to change when source changes)
- **Tools and scripts**: .github/. files (important for tooling changes)
- **Documentation**: README.md, markdown files, etc.